به نام خدا

دانشگاه تهران

دانشکده مهندسی برق و کامپیوتر

هوش مصنوعی قابل اعتماد

تمرین اول

نام و نام خانوادگی	امین آقاکثیری
شماره دانشجویی	810101381
تاریخ ارسال گزارش	1405/2/12

فهرست

سوال اول: تعمیم پذیری 4

بخش اول: سوالات توضیحی 4

زیربخش اول: مفهوم همبستگی کاذب و شکست ERM 4

زیربخش دوم: فرموله سازی خطای بدترین گروه و گروه های داده 5

زیربخش سوم: نقش تابع زیان GCE 5

زیربخش چهارم: بررسی فرض اول مقاله 6

زیربخش پنجم: ایده الگوریتم JTT 7

بخش دوم: سوالات پیاده سازی 8

زیربخش اول: آماده سازی داده و مصورسازی 8

زیربخش دوم: آموزش مدل پایه با ERM 8

زیربخش سوم: استخراج مجموعه خطا (Error Set) 9

زیربخش چهارم: ساخت دیتاست وزن دار(UPSAMPLING) 9

زیربخش پنجم: آموزش مجدد مدل 9

زیربخش ششم: پیاده سازی تابع زیان GCE 10

سوال دوم: حملات خصمانه 12

بخش اول: مرور مفاهیم 12

سوال اول 12

سوال دوم 12

سوال سوم 13

بخش دوم: حمله جعبه سیاه 13

سوال اول 13

سوال دوم 13

سوال سوم 14
سوال چهارم 14
سوال پنجم 15
بخش سوم: پیاده سازی حملات جعبه سفید 15
زیربخش اول: مدل و مجموعه داده 15
زیربخش دوم: FGSM 15
زیربخش سوم: PGD 17
مراجع 19

سوال اول: تعمیم پذیری

بخش اول: سوالات توضیحی

زیربخش اول: مفهوم همبستگی کاذب و شکست ERM

همبستگی کاذب به رابطه ای گفته می شود که در داده های آموزشی بین برچسب اصلی و یک ویژگی دیگر که bias است وجود دارد اما این رابطه واقعا علت درست تصمیم گیری نیست. در واقع مدل در داده های آموزشی مشاهده می کند که یک ویژگی ساده و قابل تشخیص، معمولا همراه با یک کلاس خاص ظاهر می شود و بنابراین ممکن است همان ویژگی را به اشتباه به عنوان نشانه اصلی آن کلاس یاد بگیرد. مثلا در مقاله گفته می شود که برای تشخیص تصوری شتر و گاو، یک نوع از این همبستگی کاذب ممکن است به این صورت باشد که تصاویر گاو در دیتاست آموزش ما بیشتر پس زمینه چمن داشته باشد و شتر بیشتر پس زمینه بیابان داشته باشد و مدل ممکن است این ویژگی (که مربوط به پس زمینه عکس ها است) را به عنوان یک همبستگی کاذب در نظر بگیرد و موقع تست هم هر جا پس زمینه چمن دید مستقل از اینکه واقعا درون تصویر گاو است یا شتر بگوید شتر در صورتی که باید به ویژگی های اصلی مانند شکل بدن و ویژگی های ظاهری حیوان توجه کند و ممکن است مثلا یک عکس جدید باشد که شتر درون یک فضای سبز است و مدل به اشتباه صرفا به دلیل پس زمینه بگوید گاو است. دلیل شکست ERM این است که هدف آن صرفا کمینه کردن خطای میانگین روی داده های آموزشی است. اگر در داده آموزش، بیشتر نمونه ها دارای همین همبستگی کاذب باشند، استفاده از ویژگی کاذب برای مدل یک مسیر ساده و کم هزینه برای کاهش خطا محسوب می شود. مثلا اگر اکثر گاو ها در تصاویر آموزشی دارای پس زمینه چمن باشند، مدل با یادگیری اینکه "پس زمینه چمن = گاو" می تواند روی بخش بزرگی از داده آموزش عملکرد خوبی به دست آورد. بنابراین از دید ERM ، تکیه کردن بر پس زمینه یک راه حل موثر برای کاهش خطای آموزشی است. ERM بین ویژگی های علی و ویژگی های صرفا همبسته فرقی نمی گذارد و فقط الگویی را ترجیح می‌دهد که خطای آموزشی را کمتر کند. از آنجا که ویژگی های کاذب معمولا راحت تر و آشکار تر و زودتر قابل یادگیری هستند، مدل به آن ها وابسته می شود و به جای یادگیری ویژگی های اصلی، روی همین نشانه های جانبی overfit می کند و درواقع شکست ERM ناشی از این است که توزیع داده آموزش نامتوازن است و نمونه های دارای همبستگی کاذب اکثریت را تشکیل می دهند.

زیربخش دوم: فرموله سازی خطای بدترین گروه و گروه های داده

در مقاله برای کاهش اثر همبستگی های کاذب، هدف آموزش به صورت یک Worst-Group Loss محاسبه می شود. به این صورت که که مدل فقط خطای میانگین کل داده ها را کم نکند بلکه روی ضعیف ترین (سخت ترین) گروه هم عملکرد خوبی داشته باشد. فرمول آن به صورت زیر است:

$$\min_{\theta} \max_{b \in \mathcal{B}} \left\{ \widehat{\mathcal{L}_b} := \frac{1}{n_b} \sum_{(x,y,b) \in G_b} \mathrm{l}(f_\theta(x), y) \right\}$$

که در واقع کاری که این تابع می کند این است که بیشترین خطا بین تمام گروه ها را کمینه می کند (که هر گروه G_b زیرمجموعه ای از داده های آموزشی متعلق به گروه b است که سایزش n_b است). در واقع تفاوتش با ERM این است که مدل نباید فقط روی گروه غالب (یا همان میانگین خطا ها) عملکرد خوبی داشته باشد بلکه باید روی گروهی که بیشترین خطا را دارد هم بهبود پیدا کند. حال دو نوع گروه اصلی در اینجا داریم. یکی نمونه های bias-aligned یا b_a هستند که در آن‌ها همبستگی کاذب رایج در داده برقرار است و ویژگی بایاس با برچسب هدف هم راستا است (مثلا در مثال سوال 1، پس زمینه نمونه گاو چمن باشد و شتر بیابان). این نمونه‌ها معمولا اکثریت داده های آموزشی را تشکیل می دهند و با ERM هم به راحتی می توان خطای آموزشی را با کمک اینها کاهش داد. دسته دیگر نمونه ها، نمونه های bias-conflicting یا b_c هستند که فاقد همبستگی کاذب رایج هستند یا با آن تضاد دارند. یعنی ویژگی بایاس با برچسب اصلی هم راستا نیست (مثلا گاو با پس زمینه بیابان). پس در فرمول Worst-Group Loss هدف این است که مدل به جای اینکه فقط خطای میانگین را کمینه کند، بیشترین خطا بین دو گروه b_a و b_c را کاهش دهد. این کار باعث می شود مدل نتواند فقط با عملکرد خوب روی نمونه های هم راستا با بایاس (b_a) به جواب خوب برسد و مجبور می شود روی نمونه های دشوارتر و در تضاد با بایاس (b_c) هم عملکرد مناسبی داشته باشد.

زیربخش سوم: نقش تابع زیان GCE

در DPR نویسندگان عمدا یک مدل سوگیر f_ϕ آموزش می دهند تا بتوانند بدون داشتن برچسب گروه های biased، حدس بزنند کدام نمونه ها احتمالا از گروه اقلیت (b_c) هستند. برای آموزش این مدل سوگیر، به جای تابع Cross-Entropy از Generalized Cross-Entropy استفاده می کنند چون هدف این نیست که مدل حتما مقاوم و تعمیم پذیر شود بلکه هدف این است که مدل تا حد امکان به همبستگی های جعلی و

ویژگی های سوگیرانه موجود در داده وابسته شود. طبق مقاله، تابع زیان GCE باعث تقویت سوگیری مدل می شود. یعنی مدل f_ϕ بیشتر به الگوهای ساده و غالب در داده، مثل ویژگی‌های bias-aligned، تکیه می کند و نسبت به نمونه های دشوارتر یا bias-conflicting حساسیت کمتری نشان می دهد. پس روی نمونه های b_a عملکرد خوبی دارد و روی b_c ها خطای بیشتری تولید می کند. پس استفاده از GCE کمک می کند مدل، سوگیر تر شود و نمونه هایی که با این سوگیری سازگار نیستند، بهتر قابل شناسایی باشند. رابطه آن به صورت زیر است:

$$l_{GCE}(f_\phi(x), y) = \frac{1 - p_\phi(y \mid x)^q}{q}$$

که در این رابطه $p_\phi(y \mid x)$ احتمالی است که مدل $f_\phi(x)$ پس از softmax به برچسب درست y برای نمونه x می دهد و q که در بازه (0,1] است یک هایپر پارامتر است. رابطه Cross-Entropy هم به صورت زیر است:

$$l_{CE}(f_\phi(x), y) = -\log p_\phi(y \mid x)$$

زیربخش چهارم: بررسی فرض اول مقاله

فرض شماره ۱ در مقاله می گوید که خطای میانگین مدل روی نمونه های bias-conflicting از خطای میانگین آن روی نمونه های bias-aligned بیشتر است. مفهوم این فرض این است که نمونه های bias-conflicting برای مدل سخت تر هستند. یعنی مدل هایی که با داده های دارای همبستگی کاذب آموزش می بینند، معمولا به جای یادگیری ویژگی های واقعی و درست، ویژگی های ساده تر و سوگیرانه را یاد می گیرند و در نتیجه روی نمونه های bias-aligned بهتر از نمونه های bias-conflicting عمل می کنند چون در آنها ویژگی سوگیری با برچسب درست هم راستا است. تابع هدف مقاله به صورت زیر بود (کمینه کردن خطای بدترین گروه):

$$\min_\theta \max_{b \in \mathcal{B}} \left\{ \widehat{\mathcal{L}_b} := \frac{1}{n_b} \sum_{(x,y,b) \in G_b} l(f_\theta(x), y) \right\}$$

که در آن $\widehat{\mathcal{L}_b}$ باید به دست آید و ببینیم کدام گروه بیشترین خطا را دارد و طبق فرض 1 مقاله می دانیم خطای روی گروه b_c بیشتر است. پس تابع هدف از حالت کلی min-max به هدفی تبدیل می شود که مستقیما روی کاهش خطای گروه b_c تمرکز دارد. اما مشکل اصلی این است که برچسب گروه ها در دسترس نیست. یعنی نمی دانیم هر نمونه متعلق به b_a است یا b_c. در اینجا فرض 1 نقش مهمی در شناسایی نمونه‌های

سخت پیدا می‌کند. چون فرض می گوید نمونه های b_c خطای بیشتری دارند، می توان نتیجه گرفت که نمونه هایی که مدل سوگیر روی آن ها دچار می شود، احتمالا متعلق به گروه b_c هستند. بنابراین الگوریتم DPR تلاش می کند احتمال عضویت نمونه در گروه b_c را تخمین بزند و سپس از این احتمال برای وزن دهی یا نمونه برداری مجدد استفاده کند. در نتیجه تابع هدف می تواند به شکل وزن دار بازنویسی شود:

$$\min_{\theta} \sum_{(x,y,b)\in\mathcal{D}} r(x,y,b)\ell(f_\theta(x),y)$$

که در آن r(x,y,b) به صورتی انتخاب می شود که نمونه هایی با احتمال بیشتر برای تعلق به گروه b_c، وزن یا احتمال نمونه برداری بیشتری بگیرند.

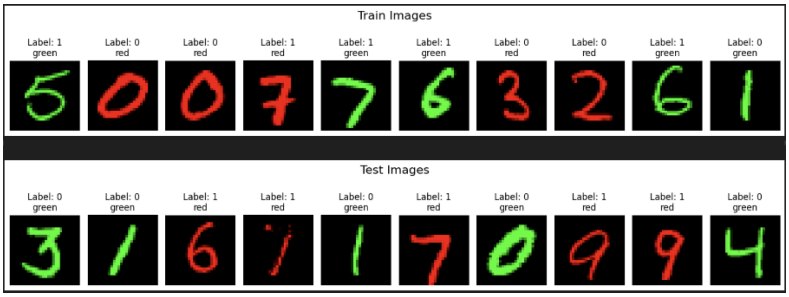
زیربخش پنجم: ایده الگوریتم JTT

ایده‌ی اصلی JTT این است که برای مقابله با همبستگی های کاذب، به جای داشتن برچسب گروهی در داده های آموزشی، از خطاهای یک مدل اولیه برای پیدا کردن نمونه های دشوار (b_c) استفاده شود. در مرحله اول، یک مدل معمولی با روش ERM آموزش داده می شود و نمونه هایی که این مدل اشتباه طبقه بندی می کند، به عنوان نمونه هایی که احتمالا متعلق به گروه های آسیب‌پذیر یا مخالف همبستگی کاذب هستند شناسایی می شوند. سپس در مرحله دوم، مدل نهایی دوباره از ابتدا آموزش داده می شود، اما این بار به نمونه های خطادار مرحله اول وزن بیشتری داده می شود. بنابراین مدل مجبور می شود به جای تکیه ساده بر ویژگی های کاذب، روی ویژگی های واقعی و تعمیم‌پذیر تر تمرکز کند و در نتیجه دقت آن در بدترین گروه ها بهبود می یابد.

بخش دوم: سوالات پیاده سازی

زیربخش اول: آماده سازی داده و مصورسازی

مراحل این بخش انجام شد. تصویر تصاویر به صورت زیر شد:



شکل 1. عکس های نمونه از دیتاست

همانطور که دیده می شود، توزیع داده ها به صورتی که گفته شده بود انجام شده است.

زیربخش دوم: آموزش مدل پایه با ERM

دقت نهایی پس از دو epoch روی داده آموزش 96.30% و روی داده تست برابر 28.65% شد. این نشان می دهد که همانطور که انتظار داشتیم، مدل به جای یادگیری ویژگی های اصلی و شکل واقعی اعداد، رنگ آنها را یاد گرفته که یک همبستگی کاذب است و به این دلیل که در داده آموزش، با احتمال 95 به 5 هر دسته از کلاس ها را رنگ آمیزی کردیم و در داده تست دقیقا برعکس اینها را و آن هم با احتمال 100 درصد رنگ آمیزی کردیم، پس به احتمال زیاد مدل در فرایند آموزش دارد همان الگوی رنگ ها را به جای ویژگی های ظاهری و واقعی اعداد یاد می گیرد و به دلیل برعکس بودن این رنگ ها در داده تست، دقت روی داده تست پایین می آید. البته مدل به صورت کامل هم بر اساس همبستگی کاذب آنها را تشخیص نداده (زیرا در آن صورت دقت 0 می شد!) اما در هر حال از یک مدل رندوم بدتر عمل کرده است. همچنین هر چه تعداد epoch ها بیشتر شود و بیشتر مدل را آموزش دهیم، مدل هم بیشتر ویژگی های واقعی را یاد می گیرد و دقت آن روی داده تست هم بالاتر می رود.

زیربخش سوم: استخراج مجموعه خطا (Error Set)

تعداد این نمونه ها برابر 2242 تا است که مطابق میزان دقت روی داده آموزش (که 96.30% شد و تعداد کل نمونه ها هم 60000 تا است) می باشد.

زیربخش چهارم: ساخت دیتاست وزن دار(UPSAMPLING)

برای این قسمت، طبق الگوریتم JTT می دانیم که پس از یک بار آموزش مدل و گرفتن دقت روی داده آموزش، وزن داده هایی که در این فرایند به اشتباه تشخیص داده شدند (که حدس می زنیم به دلیل همبستگی کاذب باشد)، بیشتر می شود و به داده های اولیه آموزش اضافه می شوند. در اینجا ما 2242 تا داده اشتباه داشتیم و ضریب تکرار هم 50 است و بنابراین در دیتاست جدید هر کدام از اینها 50 بار تکرار می شوند و به داده های قبلی (60000 تا) اضافه می شوند که می شود:

(50 * 2242) + 60000 = 172100

که در کد هم همین عدد 172100 به عنوان تعداد داده آموزشی جدید به دست آمد.

زیربخش پنجم: آموزش مجدد مدل

مدل جدید را با داده آموزشی upsample شده که در مرحله قبل ایجاد کردم آموزش دادم. دقت آن بعد از اتمام آموزش روی داده تست برابر 90.90% شد که بسیار بهتر از دقت روی داده تست در فرایند آموزش اول که برابر 28.65% شد، است. در واقع با upsample کردن، تعداد داده هایی که به خاطر همبستگی کاذب (که در اینجا رنگ است) اشتباه تشخیص داده می شدند را در داده آموزشی جدید 50 برابر کردیم و این باعث شد که در فرایند آموزش مدل دوم، به جای یادگیری و قضاوت از روی این همبستگی کاذب، مدل ویژگی های واقعی عکس ها و شکل واقعی اعداد را یاد بگیرید و برای همین دقتش روی داده تست (که اگر می خواست صرفا بر اساس همبستگی کاذب آن را برچسب بزند دقتش 0 می شد!) بسیار افزایش یافت.

زیربخش ششم: پیاده سازی تابع زیان GCE

تابع GCE نسخه تعمیم یافته ای از CE است که با پارامتر q کنترل می شود. در این سوال من مقدار q را 0.5 در نظر گرفتم. ایده اصلی GCE این است که رفتار تابع زیان نسبت به نمونه های مختلف تغییر کند و برخلاف CE، اثر بعضی نمونه های سخت یا نویزی را کمتر کند. فرمول GCE به صورت زیر است:

$$\mathcal{L}_{GCE}(x_i, y_i; \theta) = \frac{1 - f_\theta(y_i|x_i)^q}{q}$$

در این آزمایش، مدل آموزش‌دیده با GCE به دقت 95.55% روی داده آموزش و 9.31% روی داده تست رسید. این اختلاف زیاد بین دقت آموزش و تست نشان می‌دهد که مدل همچنان به همبستگی کاذب رنگ وابسته شده است. اندازه Error Set در مدل ERM برابر با 2242 نمونه بود، اما در مدل آموزش‌دیده با GCE به 2672 نمونه افزایش پیدا کرد. بنابراین استفاده از GCE باعث شد تعداد نمونه هایی که مدل روی آن ها خطا می کند بیشتر شود. همچنین نوع خطا ها هم در این دو مدل به صورت زیر بودند:

ERM Error Set analysis:

Error Set statistics:

Total errors: 2242

Class 0, Color red: 73 samples (3.26%)

Class 0, Color green: 944 samples (42.11%)

Class 1, Color red: 1123 samples (50.09%)

Class 1, Color green: 102 samples (4.55%)

GCE Error Set analysis:

Error Set statistics:

Total errors: 2672

Class 0, Color red: 0 samples (0.00%)

Class 0, Color green: 1263 samples (47.27%)

Class 1, Color red: 1395 samples (52.21%)

Class 1, Color green: 14 samples (0.52%)

می بینیم که از نظر کیفیت Error Set، نتیجه GCE بهتر و متمرکز تر به نظر می رسد. درERM ، 92.20% از نمونه‌های خطادار مربوط به حالت های ناسازگار با همبستگی رنگی بودند، یعنی کلاس 0 با رنگ سبز و

کلاس 1 با رنگ قرمز. اما درGCE ، این مقدار به 99.48% رسید و این نشان می دهد که تقریبا تمام خطا ها مربوط به نمونه های ناسازگار با همبستگی کاذب بودند. بنابراین می‌توان نتیجه گرفت که استفاده ازGCE اندازه Error Set را افزایش داده اما در عوض کیفیت آن را برای شناسایی نمونه های دشوار و ناسازگار با همبستگی کاذب بهتر کرده است. یعنی GCE باعث شده Error Set بیشتر شامل همان نمونه هایی باشد که مدل به دلیل اتکا به رنگ روی آن ها شکست می‌خورد. این موضوع نشان می‌دهد که GCE می تواند برای استخراج مجموعه خطا در مسئله های دارای همبستگی کاذب مفید باشد، هر چند به تنهایی باعث بهبود دقت آزمون نشده است.

سوال دوم: حملات خصمانه

بخش اول: مرور مفاهیم

سوال اول

در حمله جعبه‌سفید، مهاجم به اطلاعات داخلی مدل مانند معماری، گرادیان ها، وزن ها و تابع هزینه دسترسی دارد و در نتیجه می تواند با استفاده مستقیم از این اطلاعات، نمونه های متخاصم دقیق تری تولید کند. مثلا روش هایی مانند FGSM یا PGD (که در درس جزئیات آنها گفته شد) جعبه‌سفید هستند چون از گرادیان تابع خطا نسبت به ورودی استفاده می کنند. در حمله جعبه‌سیاه، مهاجم به جزئیات داخلی مدل دسترسی ندارد و فقط می تواند ورودی هایی به مدل بدهد و خروجی مدل (و یا احتمال همه برچسب های خروجی) را مشاهده کند و همچنین اغلب محدودیت تعداد ورودی ای که می توانیم به مدل بدهیم هم در این نوع از حمله وجود دارد.

سوال دوم

زیرا در دنیای واقعی معمولا مهاجم به ساختار داخلی مدل، وزن ها یا گرادیان ها دسترسی ندارد. بسیاری از مدل هایی که با آنها سر و کار داریم فقط از طریق یک API یا به صورت آنلاین (ارائه شده توسط یک شرکتی) قابل استفاده هستند و کاربر فقط خروجی مدل را می بیند. بنابراین اگر یک مدل در برابر حملات جعبه‌سفید مقاوم باشد، لزوما به این معنا نیست که در کاربرد واقعی هم مقاوم است. بررسی حملات جعبه‌سیاه کمک می کند امنیت مدل در شرایط واقعی تر ارزیابی شود و مشخص شود آیا مدل در برابر ورودی های مخرب حتی بدون دانستن جزئیات داخلی آن، آسیب پذیر هست یا نه.

سوال سوم

در حمله غیرهدف‌دار، هدف مهاجم فقط این است که مدل دچار اشتباه شود. یعنی خروجی مدل دیگر برابر با برچسب درست نباشد اما مهم نیست که برچسبی که به اشتباه توسط مدل به آن داده می شود، مربوط به کدام کلاس است. مثلا کافی است تصویر یک اسب به تصویر هر حیوان دیگری به جز اسب نسبت داده شود. اما در حمله هدف‌دار، مهاجم می خواهد مدل ورودی را دقیقا به یک کلاس مشخص و از پیش تعیین شده نسبت دهد. برای مثال، مهاجم تلاش می کند تصویر اسب حتما به عنوان تصویر سگ تشخیص داده شود. بنابراین حمله هدف‌دار معمولا سخت تر است چون علاوه بر ایجاد خطا، باید مدل را به سمت یک خروجی مشخص هدایت کند که این مسئله بهینه سازی سخت تری است.

بخش دوم: حمله جعبه سیاه

سوال اول

تصاویر طبیعی معمولا در فضای ورودیِ با ابعاد بالا، به مرزهای تصمیم مدل ها نزدیک هستند و شاید با یک تغییر بسیار کوچک بتوان برچسب نمونه را عوض کرد. در حمله جعبه‌سفید، مهاجم به اطلاعات داخلی مدل مانند گرادیان ها و وزن ها دسترسی دارد و بنابراین می تواند با استفاده از روش هایی مثل gradient descent ، مسیر مناسبی برای حرکت دادن تصویر به سمت مرز تصمیم پیدا کند. مقاله همچنین اشاره می کند که این آسیب پذیری فقط یک مشکل خاص شبکه های عصبی نیست بلکه احتمالا یک ویژگی عمومی ترِ طبقه بند ها در فضا های با ابعاد بالا است و در چنین فضا هایی، تغییرات بسیار کوچک اما هدفمند می توانند خروجی مدل را عوض کنند.

سوال دوم

دلیل استفاده از این روش این است که می خواهند یک حمله جعبه‌سیاه ساده، سریع و با تعداد پرس و جوی کم طراحی کنند. ایده کلی حمله SimBA این است که در هر مرحله، یک بردار از یک پایه متعامد انتخاب می شود و مقدار کوچکی از آن به تصویر اضافه یا از تصویر کم می شود. در واقع اگر بردار ها متعامد نباشند، ممکن است دو تغییر روی هم اثر بد بگذارند مثلا یک تغییر بخشی از تغییر قبلی را خنثی کند، یا چند تغییر

در یک جهت جمع شوند و perturbation بیش از حد بزرگ شود اما وقتی متعامد باشند، می توان اندازه نهایی تغییر را کنترل کرد. سپس با مشاهده confidence score مدل بررسی می شود که آیا احتمال کلاس درست کاهش یافته است یا نه. اگر کاهش یابد، همان تغییر نگه داشته می شود و در غیر این صورت جهت مخالف امتحان می شود. بنابراین حمله بدون نیاز به دانستن گرادیان و وزن ها یا ساختار مدل، فقط با استفاده از خروجی احتمالاتی مدل، به تدریج تصویر را به سمت مرز تصمیم حرکت می دهد.

سوال سوم

تفاوت اصلی SimBA و SimBA-DCT در فضای جست و جو بردار ها است. در SimBA ، حمله در فضای پیکسل انجام می شود. یعنی در هر مرحله مقدار یکی از مولفه های تصویر (مثلا یک پیکسل یا کانال رنگی)، کمی تغییر می کند. اما در SimBA-DCT حمله در فضای فرکانسی و با استفاده از پایه تبدیل کسینوسی گسسته انجام می شود. این روش بیشتر روی فرکانس های پایین تمرکز می کند، چون مقاله نشان می دهد تغییرات کم فرکانس معمولا برای تصاویر طبیعی موثر تر و از نظر تعداد پرس و جو بهینه تر هستند. در نتیجه، SimBA-DCT معمولا سریع تر به حمله موفق می رسد و به پرس و جو کمتری نیاز دارد اما ممکن است در برخی تصاویر به دلیل محدود شدن به فرکانس های پایین، نرخ موفقیت نهایی آن کمتر از SimBA باشد.

سوال چهارم

در روش های label-only، مهاجم فقط برچسب نهایی پیش بینی شده توسط مدل را می بیند. مثلا فقط می فهمد مدل تصویر را مثلا سگ یا اسب تشخیص داده است اما به میزان اطمینان مدل یا احتمال کلاس ها دسترسی ندارد. به همین دلیل این نوع حمله اطلاعات کمتری دارد و معمولا برای موفق شدن به تعداد پرس و جو بیشتری نیاز دارد. اما در روش های score-based، علاوه بر برچسب خروجی، امتیاز یا احتمال مربوط به کلاس ها نیز در اختیار مهاجم قرار می گیرد. این امتیازها به مهاجم کمک می کنند بفهمد تغییر اعمال شده تصویر را به مرز تصمیم نزدیک تر کرده است یا نه و بنابراین مسیر حمله بهتر هدایت می شود و معمولا تعداد پرس و جو کاهش می یابد.

سوال پنجم

این سه معیار اینها هستند: میانگین تعداد پرس و جو ها، میانگین نرم L2 اختلال ایجاد شده و نرخ موفقیت . میانگین تعداد پرس و جو ها نشان می دهد هر روش به طور متوسط چند بار باید از مدل سوال کند تا حمله موفق شود. هرچه این مقدار کمتر باشد، حمله کارآمد تر و عملی تر است. میانگین نرم L2 میزان بزرگی تغییر ایجاد شده در تصویر ورودی را اندازه می گیرد. مقدار کمتر یعنی تصویر adversarial از نظر ظاهری به تصویر اصلی نزدیک تر است و اختلال کمتر قابل مشاهده است. نرخ موفقیت نیز درصد نمونه هایی را نشان می دهد که روش توانسته با موفقیت مدل را فریب دهد. بنابراین یک حمله خوب باید هم نرخ موفقیت بالایی داشته باشد، هم با تعداد پرس و جو کم انجام شود و هم اختلال کوچکی در تصویر ایجاد کند.

بخش سوم: پیاده سازی حملات جعبه سفید

زیربخش اول: مدل و مجموعه داده

مطابق کد داده شده از قبل، با اجرای آن ، در بخش check model and dataset load تمام 25 تصویر مشاهده شد و دقت مدل هم 100% گزارش شد یعنی مدل ResNet-20 توانست تمامی تصاویر را به درستی طبقه بندی کند.

زیربخش دوم: FGSM

روش FGSM یا Fast Gradient Sign Method یکی از ساده ترین و سریع ترین روش‌های حمله متخاصم در حالت جعبه‌سفید است. ایده اصلی FGSM این است که تصویر ورودی را در جهتی کمی تغییر دهیم که خطای مدل بیشترین افزایش را پیدا کند. چون این تغییر بسیار کوچک است، تصویر برای انسان تقریبا همان تصویر قبلی دیده می شود، اما ممکن است مدل آن را اشتباه طبقه بندی کند. فرمول حمله Non-Targeted FGSM به صورت زیر است:

$$x' = x + \epsilon \text{ sign } [\nabla_x J(\theta, x, y)]$$

در این فرمول، x تصویر اصلی، x' تصویر متخاصم، ϵ اندازه تغییر یا شدت حمله، $J(\theta, x, y)$ تابع خطای مدل، θ پارامترهای مدل، و y برچسب اصلی تصویر است. عبارت $\nabla_x J(\theta, x, y)$ گرادیان خطا نسبت به ورودی

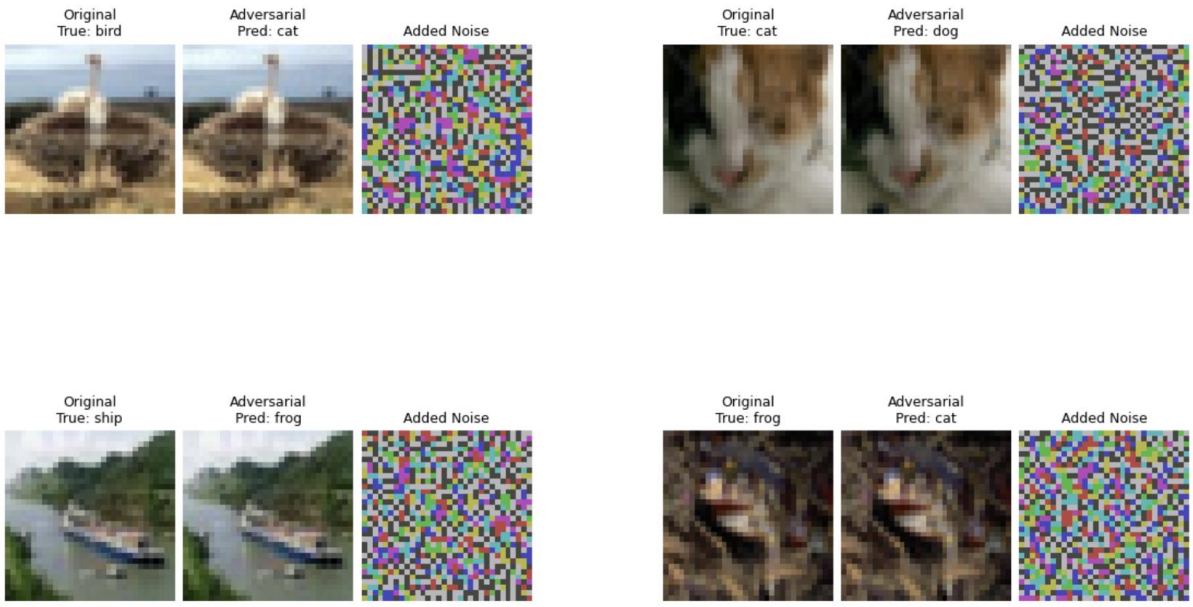
را نشان می دهد. یعنی مشخص می کند اگر هر پیکسل کمی تغییر کند، خطای مدل چقدر تغییر می کند. تابع sign فقط علامت گرادیان را نگه می دارد و مشخص می کند هر پیکسل باید کمی زیاد یا کم شود. بنابراین FGSM با یک گام سریع، ورودی را در جهت افزایش خطا تغییر می دهد. در واقع تصویر ورودی را به گونه ای تغییر می دهد که تابع خطا (که خطای بین ورودی و هر برچسبی غیر از برچسب درست را محاسبه می کند) افزایش پیدا کند.

برای حمله Targeted، هدف این نیست که مدل فقط اشتباه کند، بلکه می خواهیم مدل تصویر را به یک کلاس مشخص y' نسبت دهد. در این حالت فرمول به شکل زیر نوشته می‌شود:

$$x' = x - \epsilon \text{ sign } [\nabla_x J(\theta, x, y')]$$

در اینجا y' کلاس هدف است. علامت منفی به این دلیل استفاده می‌شود که می‌خواهیم خطای مدل نسبت به کلاس هدف کم شود. یعنی مدل با اطمینان بیشتری تصویر را به عنوان آن کلاس هدف تشخیص دهد. پس در کل FGSM با استفاده از گرادیان، یک نویز کوچک اما جهت دار به تصویر اضافه می‌کند. این نویز برای انسان معمولا قابل تشخیص نیست، اما چون دقیقا در جهتی اعمال می‌شود که مدل را دچار خطا می‌کند، می‌تواند باعث تغییر خروجی مدل شود.

مطابق فرمول گفته شده برای Non-Targeted، FGSM را پیاده سازی کردم. دقت مدل روی این تصاویر جدید که FGSM روی آنها اعمال شد برابر 28% شد که یعنی 18 نمونه از 25 نمونه به اشتباه طبقه بندی شدند و این حمله توانست به دقت مدل بسیار آسیب بزند. 4 نمونه از تصاویری که مدل در تشخیص آن ها دچار خطا شد به صورت زیر هستند:



شکل 2. چهار نمونه از تصویر اصلی، تصویر خصمانه، نویز افزوده شده، برچسب واقعی و برچسب اشتباه پیشبینی شده برای FGSM

همانطور که در تصویر مشاهده می شود، این نوع FGSM به دلیل Non-Targeted بودن، هدفش این بود که برچسب تصویر تولید شده توسط مدل هر چیزی غیر از برچسب درستش طبقه بندی شود که این اتفاق افتاد.

زیربخش سوم: PGD

PGD یا Projected Gradient Descent در واقع نسخه چند مرحله ای FGSM است. در FGSM فقط یک بار در جهت افزایش خطای مدل حرکت می کنیم اما در PGD این کار در چند گام کوچک تکرار می‌شود تا نمونه متخاصم قوی تری ساخته شود. فرمول حمله Non-Targeted برای PGD به صورت زیر است:

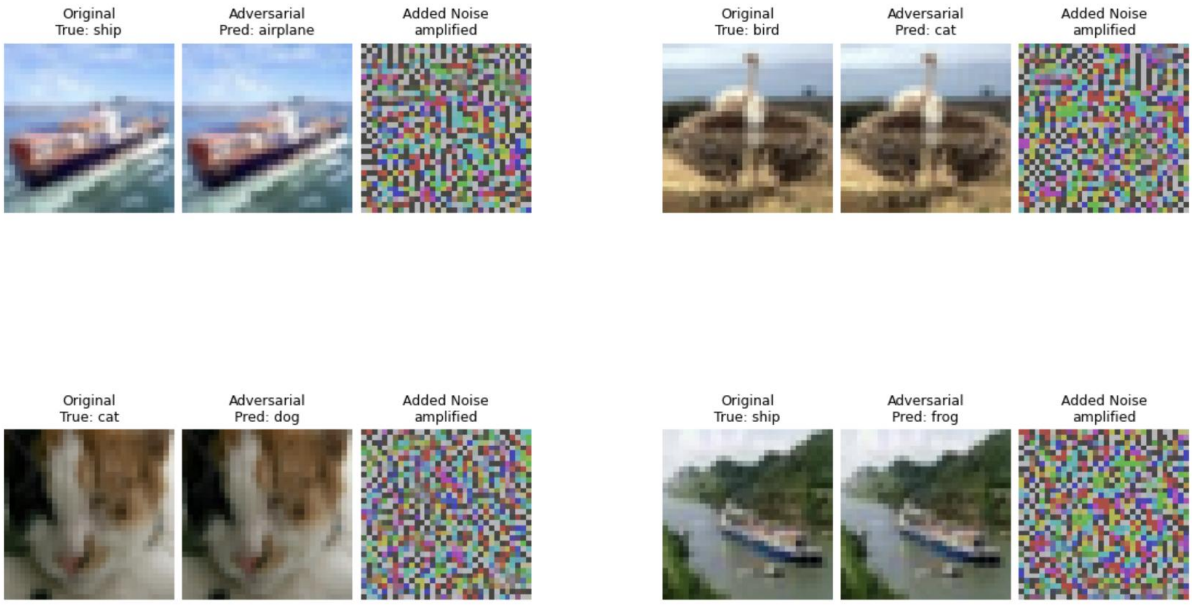
$$x'_{t+1} = \text{Proj}\ \{x'_t + \alpha\ \text{Sign}\ [\nabla_x J\ (\theta, x'_t, y)]\}$$

در این فرمول، x'_t تصویر متخاصم در مرحله t، x'_{t+1} تصویر متخاصم در مرحله بعدی اجرای الگوریتم، $J(\theta, x'_t, y)$ تابع خطای مدل، θ پارامترهای مدل، و y برچسب واقعی اصلی است. پارامتر α اندازه گام در هر مرحله است و مشخص می کند در هر تکرار چقدر تصویر تغییر کند. وقتی گرادیان loss را نسبت به ورودی حساب می کنیم، یعنی $\nabla_x J$، متوجه می شویم اگر هر پیکسل تصویر کمی تغییر کند، خطای مدل در چه جهتی بیشتر می شود. سپس با استفاده از sign فقط جهت مثبت یا منفی تغییر هر پیکسل را نگه می داریم نه اندازه دقیق گرادیان را. در نتیجه، عبارت $\alpha\ \text{Sign}\ [\nabla_x J\ (\theta, x'_t, y)]$ یک تغییر کوچک و جهت دار به تصویر اضافه می کند. همچنین یک پارامتر ϵ هم داریم که حداکثر فاصله مجاز بین تصویر اصلی و تصویر متخاصم است. Proj به معنی تصویر کردن دوباره نمونه داخل ناحیه مجاز است. یعنی اگر بعد از یک گام، تصویر متخاصم بیش از حد از تصویر اصلی دور شود، دوباره به داخل محدوده ای برگردانده می شود. اگر حمله با نرم L_∞ تعریف شده باشد، این کار معمولا یعنی مقدار هر پیکسلِ تصویر متخاصم حداکثر به اندازه ϵ از مقدار همان پیکسل در تصویر اصلی فاصله داشته باشد. همچنین معمولا بعد از هر مرحله مقدار پیکسل ها به بازه معتبر تصویر (مثلا [0,1]) محدود می شود. این کار باعث می شود تغییرات تصویر همچنان کوچک و معمولا برای انسان نامحسوس باقی بماند. همچنین مقدار دهی اولیه x_0 از خود x که عکس ورودی اصلی است شروع می شود. البته می توان به ورودی اولیه یک نویز کوچک δ هم اضافه کرد و سپس $x+\delta$ را به عنوان x_0 داد. حالت حمله Targeted هم برای PGD مثل FGSM است و فرمول آن به صورت زیر است:

$$x'_{t+1} = \text{Proj}\ \{x'_t - \alpha\ \text{Sign}\ [\nabla_x J\ (\theta, x'_t, y')]\}$$

که در اینجا y' کلاس هدف است. علامت منفی به این دلیل استفاده می‌شود که می‌خواهیم خطای مدل نسبت به کلاس هدف کم شود. یعنی مدل با اطمینان بیشتری تصویر را به عنوان آن کلاس هدف تشخیص دهد.

مطابق فرمول گفته شده برای Non-Targeted و هایپر پارامتر های گفته شده، PGD را پیاده سازی کردم. دقت مدل روی این تصاویر جدید که PGD روی آنها اعمال شد برابر 4% شد که یعنی فقط 1 نمونه از 25 نمونه به درستی طبقه بندی شده و این حمله از FGSM هم بسیار بهتر عمل کرده است. 4 نمونه از تصاویری که مدل در تشخیص آن ها دچار خطا شد به صورت زیر هستند:



شکل 3. چهار نمونه از تصویر اصلی، تصویر خصمانه، نویز افزوده شده، برچسب واقعی و برچسب اشتباه پیشبینی شده برای PGD

در واقع مشخص شد که حمله PGD نسبت به FGSM عملکرد قوی تری دارد. دلیل اصلی این تفاوت آن است که FGSM تنها در یک گام و با استفاده از علامت گرادیان، نویز خصمانه را تولید می کند اما PGD این فرآیند را در چندین گام کوچک تر انجام می دهد و پس از هر گام، تصویر را دوباره به محدوده مجاز نویز بر می گرداند. به همین دلیل PGD فرصت بیشتری برای پیدا کردن جهت موثر ترِ تغییر تصویر دارد.

مراجع

[1]E. Z. Liu, B. Haghgoo, A. S. Chen, A. Raghunathan, P. W. Koh, S. Sagawa, P. Liang, and C. Finn, “Just Train Twice: Improving Group Robustness without Training Group Information,” Proceedings of the 38th International Conference on Machine Learning (ICML), PMLR 139, 2021.

[2]H. Han, S. Kim, H. Joo, S. Hong, and J. Lee, “Mitigating Spurious Correlations via Disagreement Probability,” Advances in Neural Information Processing Systems (NeurIPS), 2024.

[3]C. Guo, J. R. Gardner, Y. You, A. G. Wilson, and K. Q. Weinberger, “Simple Black-box Adversarial Attacks,” Proceedings of the 36th International Conference on Machine Learning (ICML), PMLR 97, 2019.